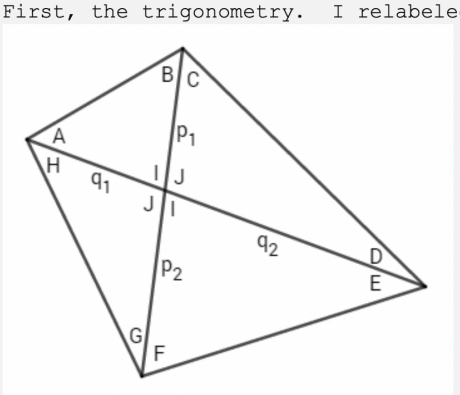
First, the trigonometry. I relabeled everything like so:



There are 10 angles in all, including the two for the intersection of the diagonals, but only 4 can be varied, as well as one of the semi-diagonal arm lengths, forcing the rest.

* Choose I, which forces J
* Choose A, which with I forces B
* Choose p_1, which with A and B forces q_1
* Choose C, which with J forces D and with p_1 forces q_2
* Choose E, which with I forces F and with q_2 forces p_2
* G and H are now forced - how is the question.

Use the law of sines to derive an equation relating the 8 outside angles:

$$\frac{p_1}{\sin A} = \frac{q_1}{\sin B} \implies q_1 = p_1 \frac{\sin B}{\sin A}$$
$$\frac{q_2}{\sin C} = \frac{p_1}{\sin D} \implies q_2 = p_1 \frac{\sin C}{\sin D}$$
$$\frac{p_2}{\sin E} = \frac{q_2}{\sin F} \implies p_2 = q_2 \frac{\sin E}{\sin F} = p_1 \frac{\sin C \sin E}{\sin D \sin F}$$
$$\frac{q_1}{\sin G} = \frac{p_2}{\sin H} \implies p_1 \frac{\sin B}{\sin A \sin G} = p_1 \frac{\sin C \sin E}{\sin D \sin F \sin H}$$
$$\implies \frac{\sin H}{\sin G} = \frac{\sin A \sin C \sin E}{\sin B \sin D \sin F}$$

That last is what I need, but there's a more beautiful form:

$$\sin A \sin C \sin E \sin G = \sin B \sin D \sin F \sin H$$

Now use the identity for the sine of the difference of two angles to get:

$$\sin H = \sin(I - G) = \sin I \cos G - \cos I \sin G$$

Divide that through by $\sin G$ and substitute into the end of that chain of derivations, rearrange, and arrive at:

$$G = \cot^{-1}\left(\frac{\sin A \sin C \sin E}{\sin B \sin D \sin F \sin I} + \cot I\right)$$

With that, you can easily test whether a given choice for the free angles I, A, C, and E results in an integral G.

I implemented a couple optimizations. First, I precalculated the sines for integer degrees from 0 to 180, which cut the running time by about 40%. Second, I limited the range of the free variables to avoid creating a bunch of similar quadrilaterals:

$I \in [2, 90]$, $A \in [1, J-1]$, $C \in [1, \lfloor \frac{I}{2} \rfloor]$, $E \in [A, J-1]$

That cut the number of duplicates down from 75% of the integer quadrilaterals discovered to just 1% (almost all of which occur when the diagonals are perpendicular).

Running time is about 0.8 secs in pypy3, ten times that in CPython 3.6, testing 14,792,295 quadrilaterals to find the 129,325 solutions and 1,295 similar non-solutions.